Referencia HTTP de MDN: códigos de estado y encabezados (selección)

100 Continue

El código de respuesta de estado informativo 100 Continue indica que todo hasta ahora está bien y que el cliente debe continuar con la solicitud o ignorarlo si ya está terminado.

Para que un servidor verifique los encabezados de la solicitud, un cliente debe enviar Expect: 100-continue como encabezado en su solicitud inicial y recibe un código de estado 100 Continue en respuesta antes de enviar el cuerpo.

Estado

100 Continue

Especificaciones

Compatibilidad con navegadores

Ver también

- Expect
-

101 Switching Protocols

El código de respuesta 101 Switching Protocols que el servidor está cambiando de protocolo al solicitado por un cliente que mandó un mensaje incluyendo la cabecera Upgrade.

El servidor incluye en esta respuesta una cabecera Upgrade para indicar a qué protocolo ha cambiado. El proceso se describe en detalle en el artículo Protocol upgrade mechanism.

Estado

101 Switching Protocols

Ejemplos

El cambio de protocolos se podría usar con WebSockets.

HTTP/1.1 101 Switching Protocols Upgrade: websocket Connection: Upgrade

Especificaciones

Specification	Title
	Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content

Vea también

- Protocol upgrade mechanism
- WebSockets
- Upgrade

- 426 Upgrade Required

200 OK

El código de respuesta de estado satisfactorio HTTP 200 OK indica que la solicitud ha tenido éxito. Una respuesta 200 es almacenable de forma predeterminada.

El significado de un éxito depende del método de solicitud HTTP:

- GET: El recurso ha sido recuperado y se transmite el mensaje al body.
- HEAD: Los encabezados de entidad estan en el body del mensaje.
- POST: El recurso que describe el resultado de la acción se transmite en el body del mensaje.
- TRACE: El body del mensaje contiene el mensaje de solicitud tal como lo recibió el servidor.

El resultado exitoso de un método PUT o uno DELETE no es a menudo un 200 OK sino un 204 No Content (o un 201 Created cuando el recurso es subido por primera vez).

Estado

200 OK

Especificaciones

Compatibilidad con navegadores

Ver también

- HTTP request methods

201 Created

El código de respuesta de estado de éxito creado HTTP 201 Created indica que la solicitud ha tenido éxito y ha llevado a la creación de un recurso. El nuevo recurso se crea efectivamente antes de enviar esta respuesta. y el nuevo recurso se devuelve en el cuerpo del mensaje, su ubicación es la URL de la solicitud o el contenido del encabezado de la Ubicacion

El caso de uso común de este código de estado es el resultado de una solicitud metodo POST

Status

201 Created

Especificaciones

Compatibilidad con navegadores

Ver también

- HTTP request methods

202 Aceptado

El código de respueta de estado del Protocolo de Transferencia de Hipertexto (HTTP) 202 Aceptado indica que la petición ha sido recibida pero que todavía no se ha actuado al respecto. Es libre, en el sentido de que no hay manera para el HTTP para enviar después una respuesta asíncrona indicando el resultado del procesamiento de la petición. Es pretendida para casos donde otro proceso o servidor maneje la petición, o para procesamiento por lotes.

Estado

202 Aceptado

Especificaciones

Specification	Title
	Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content

Ver también

- Accept

203 Non-Authoritative Information

El código de respueta de estado del Protocolo de Transferencia de Hipertexto (HTTP) 203 Non-Authoritative Information indica que la peticion fue satisfactoria pero su contenido ha sido modificado por un transformador Proxy server desde los origenes del servidor 200 (OK)

El código de respuesta 203 es similar al código 214, quiere decir Transformation Applied, of the Warning header code, que tiene la ventaja adicional de estar disponible para las respuestas con cualquier código.

Status

203 Non-Authoritative Information

Specifications

Specification	Title
	Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content

See also

- 200
- Proxy server
- Warning

206 Partial Content

El codigo de respuesta con estado exitoso HTTP 206 Partial Content indica que la solicitud se ha realizado con exito y el cuerpo contiene los rangos solicitados de la data, como esta descrito

en la cabecera Range de la solicitud.

Si solo hay un rango, el Content-Type de toda la respuesta es asignada a un tipo de documento, y un Content-Range es provisto.

Si muchos rangos son retornados, el Content-Type es asignado a multipart/byteranges y cada fragmento cubre un rango, con Content-Range y Content-Type describiendolo .

Status

206 Partial Content

Ejemplos

Una respuesta conteniendo un solo rango:

HTTP/1.1 206 Partial Content Date: Wed, 15 Nov 2015 06:25:24 GMT Last-Modified: Wed, 15 Nov 2015 04:58:08 GMT Content-Range: bytes 21010-47021/47022 Content-Length: 26012 Content-Type: image/gif

... 26012 bytes of partial image data ...

Una respuesta conteniendo varios rangos:

HTTP/1.1 206 Partial Content Date: Wed, 15 Nov 2015 06:25:24 GMT Last-Modified: Wed, 15 Nov 2015 04:58:08 GMT Content-Length: 1741 Content-Type: multipart/byteranges; boundary=String_separator

--String_separator
Content-Type: application/pdf
Content-Range: bytes 234-639/8000

...the first range...
--String_separator
Content-Type: application/pdf
Content-Range: bytes 4590-7999/8000

...the second range
--String_separator--

Especificaciones

Compatibilidad con navegadores

Mira también

- If-Range
- Range
- Content-Range
- Content-Type

302 Found

El código de estado de redirección HTTP 302 Found indica que el recurso solicitado ha sido movido temporalmente a la URL dada por las cabeceras Location. Un navegador redirecciona a esta página, pero los motores de búsqueda no actualizan sus enlaces al recurso (hablando en lenguaje SEO, se suele decir que el link juice no es enviado a la nueva URL).

Incluso si la especificación requiere el método, y el cuerpo, no debe ser alterado cuando la redirección se completa, no todos los user-agents se conforman aquí, y tu puedes encontrar software inestable por ahí. Por la tanto se recomienda poner el código 302 sólo como respuesta a los métodos GET o HEAD , y usar en cambio 307 Temporary Redirect , ya que el método de cambio está explicitamente prohibido en ese caso.

En casos en los que quieras que el método usado para cambiar a GET, usa 303 See Other. Esto es práctico cuando quieres dar una respuesta al método PUT que no es el recurso subido, pero sí un mensaje de confirmación (como "Has subido satisfactoriamente XYZ").

Estado

302 Found

Especificaciones

Compatibilidad con navegadores

Mira también

- 307 Temporary Redirect, el equivalente a este código de estado, pero que nunca cambia el método usado.
- 303 See Other, una redirección temporal que cambia el método usado a GET.
- 301 Moved Permanently, la redirección permanente.

304 Not Modified

El código HTTP de redirección 304 Not Modified en el response de la petición indica que no hay necesidad de retransmitir los recursos solicitados. Es una redirección implícita a un elemento/recurso de caché.Esto sucede cuando el método de la solicitud es (), como en el las peticiones con métodos GET o HEAD, o cuando el request (petición) está condicionada y usa la cabecera If-None-Match o un If-Modified-SinceEl response 200 OK habría incluido los encabezados Cache-Control, Content-Location, Date, ETag, Expires, y Vary.

NOTE: Muchos developer tools' network panels (paneles de red de desarrollo) de los navegadores crean extraños request que conducen a un "response(respuesta del servidor) 304 ", entonces el acceso al caché local es accesible a los desarrollodares.

Status

304 Not Modified

Especificaciones

Compatibilidad con navegadores

See also

- If-Modified-Since
- If-None-Match

400 Petición mala

La respuesta de código de estado del Protocolo de Transferencia de Hipertexto (HTTP) 400 Bad Request indica que el servidor no puede o no procesará la petición debido a algo que es percibido como un error del cliente (p. ej., sintaxis de petición malformada, solicitud inválida de enmarcado de mensajes, o enrutamiento engañoso de peticiones).

WARNING: El cliente no debería repetir esta petición sin modificarla.

Estado

400 Petición mala

Specifications

Specification	Title
	Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content

401 Unauthorized

El código de error HTTP 401 indica que la petición (request) no ha sido ejecutada porque carece de credenciales válidas de autenticación para el recurso solicitado.

Este estatus se envia con un WWW-Authenticate encabezado que contiene informacion sobre como autorizar correctamente.

Es similar al estatus 403, pero en este caso , la autenticación si es posible.

Estado

401 Unauthorized

Respuesta de ejemplo

HTTP/1.1 401 Unauthorized Date: Wed, 21 Oct 2015 07:28:00 GMT WWW-Authenticate: Basic realm="Access to staging site"

Especificaciones

Compatibilidad con navegadores

Vea también

- HTTP authentication
- WWW-Authenticate
- Authorization

- Proxy-Authorization
- Proxy-Authenticate
- 403, 407

403 Forbidden

El error 403 o 403 Forbidden es un código de respuesta HTTP el cual indica que el servidor ha recibido y ha entendido la petición, pero rechaza enviar una respuesta.

Hay similitudes entre el status 401 y el error 403, la diferencia es que este último no se soluciona con una re-autenticación. El acceso está permanentemente prohibido y ligado a la lógica de la aplicación, como el no tener los permisos necesarios para acceder al recurso.

Estado

403 Forbidden

Ejemplo de respuesta

HTTP/1.1 403 Forbidden Date: Wed, 21 Oct 2015 07:28:00 GMT

Especificaciones

Compatibilidad de navegadores

Vea también

- 401
- HTTP/1.1: Status Code Definitions

404 Not Found

El codigo de error HTTP 404 Not Found (404 No Encontrado) de respuesta de cliente indica que el servidor no puede encontrar el recurso solicitado. Vinculos que conducen a una pagina 404 son normalmente llamados vinculos rotos o vinculos muertos, y pueden estar sujetos a Enlace Roto.

Un código de estado 404 no indica si el recurso está temporalmente o permanentemente ausente. Pero si un recurso es permanentemente eliminado, un (Gone) debe ser usado en lugar del estado 404.

Estado

404 Not Found

En español:

404 No Encontrado

Paginas de error personalizadas

Muchos sitios web personalizan la apariencia de la pagina 404 para que sea de utilidad al usuario y proveen una guia para saber qué hacer. Servidores Apache pueden ser configurados

usando un archivo .htaccess con el siguiente codigo:

ErrorDocument 404 /no-encontrado.html

Para una pagina 404 de ejemplo, mire la pagina MDN 404.

NOTE: Diseños personalizados son buenos, si se usan de manera moderada. Siente libre de hacer tus paginas 404 humoristicas y humanas, pero no confundas a tus usuarios.

Especificaciones

Compatibilidad con navegadores

Vea también

-
- Wikipedia: HTTP 404

405 Method Not Allowed

El código de estado de respuesta 405 Method Not Allowed (Método no permitido, en español) del protocolo de transferencia de hipertexto (HTTP) indica que el servidor conoce el método de solicitud, pero el recurso de destino no admite este método.

El servidor debe generar un campo de cabecera Allow en una respuesta con código de estado 405. El campo debe contener una lista de métodos que el recurso de destino admite actualmente.

Estado

405 Method Not Allowed

Especificaciones

Véase también

- Allow
- Definiciones de código de estado HTTP
- Cómo arreglar 405 Method Not Allowed
- Solución de problemas de HTTP 405

408 Request Timeout

El código de estado de la respuesta 408 Request Timeout del Protocolo de Transferencia de Hipertexto (HTTP) significa que el servidor desea cerrar esta conexión no usada. Se envía a una conexión inactiva por algunos servidores, incluso sin solicitud previa por parte del cliente.

Un servidor debe enviar "close" en el campo de la cabecera Connection en la respuesta, ya que 408 implica que el servidor ha decidido cerrar la conexión en lugar de continuar esperando.

Esta respuesta es usada mucho más desde que algunos navegadores, como Chrome, Firefox 27+, y IE9, usan el mecanizmo de pre-conexión HTTP para acelerar la naveación.

NOTE: Algunos servidores simplemente cierran la conexión sin enviar este mensaje.

Estado

408 Request Timeout

Especificaciones

Especificación	Título
	Protocolo de Transferencia de HiperTexto (HTTP/1.1): Semánticas y Contenido

Ver también

- Connection
- X-DNS-Prefetch-Control

413 Content Too Large

El código de estado de respuesta HTTP 413 Content Too Large indica que la entidad de solicitud es mayor que los límites definidos por el servidor; el servidor podría cerrar la conexión o devolver un campo de cabecera Retry-After.

Antes del RFC 9110, la frase de respuesta para el estado era Payload Too Large. Ese nombre todavía se usa ampliamente.

Estado

413 Content Too Large

Especificaciones

Véase también

- Connection
- Retry-After

418 Soy una tetera

El código de error HTTP 418 Soy una tetera indica que el servidor se rehusa a preparar café porque es una tetera. Este error es una referencia al Hyper Text Coffee Pot Control Protocol, creado como parte de una broma del April Fools' de 1998.

Estado

418 I'm a teapot

Especificaciones

Compatibilidad con navegadores

Referencia HTTP de MDN: códigos de estado y encabezados (selección)

Ver también

- Wikipedia: Hyper Text Coffee Pot Control Protocol (Español)
- Wikipedia: Hyper Text Coffee Pot Control Protocol (Inglés)

500 Internal Server Error

El código de respuesta de error del servidor del Protocolo de transferencia de hipertexto (HTTP) 500 Internal Server Error (Error interno de servidor 500) indica que el servidor encontró una condición inesperada que le impidió cumplir con la solicitud.

Esta respuesta de error es una respuesta genérica "atrapa todo". Por lo general, esto indica que el servidor no puede encontrar un mejor código de error 5xx para responder. A veces, los administradores del servidor registran respuestas de error como el código de estado 500 con más detalles sobre la solicitud para evitar que el error vuelva a ocurrir en el futuro.

Estado

500 Internal Server Error

Especificaciones

Compatibilidad con navegadores

Véase también

- Definiciones de código de estado HTTP

502 Puerta de enlace no válida

El código de respuesta de error del servidor de HTTP 502 Bad Gateway indica que el servidor, mientras actuaba como una puerta de enlace o proxy, recibió una respuesta no válida del servidor ascendente.

NOTE: Una puerta de enlace puede referirse a cosas distintas en redes y un error 502 no es algo que normalmente puedas arreglar, ya que requiere correcciones por parte del servidor o los proxies a través de los que intentas acceder.

Estado

502 Bad Gateway

Especificaciones

Compatibilidad con navegadores

Vea también

-
- 502 Bad Gateway

503 Servicio No Disponible

El envío de un código de error 503 Servicio No Disponible como respuesta por un servidor que use el Protocolo de Transferencia de Hipertexto (HTTP) indica que el servidor no está listo para manejar la solicitud.

Las causas más comunes son que el servidor esté apagado por mantenimiento o esté sobrecargado. Esta respuesta debería usarse para condiciones temporales y la cabecera HTTP Retry-After debería, si es posible, contener el tiempo estimado para la recuperación del servicio.

NOTE: Debería enviarse con esta respuesta una página informativa explicando el problema.

Debe tenerse cuidado con las cabeceras relacionadas con la caché, ya que un estado 503 suele ser algo temporal, y, por lo tanto, no se deberían almacenar las respuestas en caché.

Estado

503 Servicio No Disponible

Especificaciones

Compatibilidad con navegadores

Ver también

- Retry-After

504 Gateway Timeout

El código de respuesta de error del servidor de HTTP 504 Gateway Timeout indica que el servidor, mientras actuaba como una puerta de enlace o proxy, no pudo obtener una respuesta a tiempo.

NOTE: Una puerta de enlace puede referirse a cosas distintas en redes y un error 502 no es algo que normalmente puedas arreglar, ya que requiere correcciones por parte del servidor o los proxies a través de los que intentas acceder.

Estado

504 Gateway Timeout

Especificaciones

Compatibilidad con navegadores

Vea también

-

505 HTTP Version Not Supported

El código de estado de respuesta del Protocolo de transferencia de hipertexto (HTTP, por sus siglas en inglés) 505 HTTP Version Not Supported indica que la versión HTTP utilizada en la solicitud no es compatible con el servidor.

Estado

505 HTTP Version Not Supported

Especificaciones

Véase también

- Upgrade

Cache-Control

El encabezado HTTP Cache-Control contiene directivas (instrucciones) — tanto en peticiones como en respuestas — para controlar el almacenamiento temporal (caching) en navegadores y cachés compartidas (p. ej. Proxies, CDNs).

<tbody>

Tipo de encabezado |

Request header,
Response header
|

Forbidden header name
|
no |

CORS-safelisted response header
|
sí |

</tbody>

Sintaxis

Las directivas para almacenamiento temporal siguen las siguientes reglas de validación:

- Insensible a mayúsculas, — pero las minúsculas son recomendadas, debido a que algunas implementaciones no reconocen las directivas en mayusculas.
- Las multipes directivas son separadas por comas.
- Algunas directivas tienen un argumento opcional.

Directivas de cache

Las directivas Cache-Control estándar están definidas a continuación.

Request	Response
max-age	max-age
max-stale	-

min-fresh	-
-	s-maxage
no-cache	no-cache
no-store	no-store
no-transform	no-transform
only-if-cached	-
-	must-revalidate
-	proxy-revalidate
-	must-understand
-	private
-	public
-	immutable
-	stale-while-revalidate
stale-if-error	stale-if-error

Nota: Comprueba la tabla de compatibilidad para su soporte; los agentes de usuario que no las reconozcan deberían ignorarlas.

Vocabulario

Los siguientes términos son usados en este documento; algunos provienen de la especificación.

- Caché (HTTP)
Implementación que mantiene peticiones y respuestas para reusarlas en peticiones posteriores. Puede ser tanto una cache privada como una compartida.
- Caché compartida
Caché existente entre el servidor de origen y los clientes (p. ej. Proxy, CDN). Almacena una sola respuesta para reutilizarla en multiples usuarios — por tanto los desarrolladores deberían evitar que el almacenamiento de contenidos personalizados sea cacheado en la caché compartida.
- Caché privada
Caché existente en el cliente. También conocida como caché local, o caché del navegador. Puede almacenar y reutilizar contenido personalizado para un único usuario.
- Respuesta almacenada
Almacena una respuesta en caches cuando es cacheable. Pero no siempre es reutilizada tal cual. (Normalmente "caché" significa almacenar una respuesta).
- Respuesta reutilizada
Reutiliza respuestas cacheadas para las subsiguientes peticiones.
- Revalidar respuesta
Pregunta al servidor de origen si una respuesta almacenada sigue siendo reciente o no (fresh o stale). Normalmente se realiza a través de una petición condicionada.
- Respuesta reciente
Indica que una respuesa es reciente (fresh). Esto normalmente significa que la respuesta puede ser reutilizada para las subsiguientes peticiones, dependiendo de las directivas de petición.
- Respuesta obsoleta
Indica que la respuestá está obsoleta (stale). Normalmente significa que la respuesta ya no puede ser reutilizada. El almacenamiento caché no requiere que las respuestas obsoletas sean eliminadas inmediatamente, por que la revalidación puede cambiar la respuesta de obsoleta a reciente de nuevo.
- Edad
El tiempo desde que una respuesta fue generada. Es un criterio para ver si una respuesta es reciente u obsoleta (fresh o stale).

Directivas

Esta sección lista directivas que afectan al almacenamiento caché — directivas de respuestas y directivas de peticiones.

Directivas de Respuesta

max-age

La directiva de respuesta max-age=N indica que la respuesta es reciente hasta los N segundos posteriores a su generación.

Cache-Control: max-age=604800

Indica que las cachés pueden almacenar esta respuesta y reutilizarla para las peticiones subsecuentes mientras estas son recientes.

Ten en cuenta que max-age no es el tiempo pasado desde que la respuesta fue recibida, sino el tiempo desde que la respuesta fue generada en el servidor de origen. Por tanto si otra(s) caché(s) — en la ruta de red de la respuesta — la almacenan por 100 segundos (indicado usando el campo de la cabecera Age en la respuesta), el navegador descontará 100 segundos del periodo de validez de la caché de respuesta.

Cache-Control: max-age=604800 Age: 100

s-maxage

La directiva de respuesta s-maxage también indica por cuánto tiempo la respuesta es reciente (similar a max-age) — pero es específica para cachés compartidas, e ignorarán max-age cuando está presente.

Cache-Control: s-maxage=604800

no-cache

La driectiva de respuesta no-cache indica que la respuesta puede ser almacenada en cachés, pero debe ser validada con el servidor de origen antes de cada reutilización — incluso cuando la caché está desconectada del servidor de origen.

Cache-Control: no-cache

Si quieres que las cachés siempre comprueben la actualización de contenido cuando reúsen el contenido almacenado, no-cache es la directiva a usar. Esta obliga a la cache a revalidarla con cada petición al servidor de origen.

Ten en cuenta que no-cache no significa "no almacenar". no-cache permite almacenar una respuesta, pero les obliga a revalidarla antes de reusarla. En caso de que "no almacenar" sea lo que estabas buscando, entonces no-store es la directiva a usar.

must-revalidate

La directiva must-revalidate indica que la respuesta puede ser usada mientras sea reciente, pero que una vez el recurso se vuelve obsoleto, la cache no debe usar su copia obsoleta sin correctamente validar en el servidor de origen.

Tipicamente, must-revalidate es usada con max-age

Cache-Control: max-age=604800, must-revalidate

HTTP permite a las caches reutilizar respuesteas obsoletas cuando están desconectados del servidor de origen. must-revalidate es una forma de prevenirlo, o la cache revalida la respuesta almacenada con el servidor de origen, o si no es posible genera una respuesta 504 (Gateway Timeout).

proxy-revalidate

proxy-revalidate es similar a must-revalidate, pero es especifico para caches compartidos.

no-store

La directiva de respuesta no-store indica que cualquier caché de cualquier tipo (privado o compartido) no debe almacenar esta respuesta.

Cache-Control: no-store

private

La directiva de respuesta private indica que la respuesta solo puede ser almacenada por cachés privadas (p. ej. cachés locales en navegadores).

Cache-Control: private

Deberías añadir la directiva private para el contenido personalizado de usuario — en particular, respuestas recibidas después del login, y sesiones administradas via cookies.

Si olvidas añadir private a una respuesta con contenido personalizado, entonces esa respuesta puede ser almacenada en una caché compartida y terminar siendo reutilizada por multiples usuarios, lo cual puede causar una fuga de información personal.

public

La directiva public indica que la respuesta puede ser almacenada en un cache compartido. Las respuestas para peticiones con el campo de la cabecera Authorization no debe ser almacenadas en cache compartida, pero la directiva public causará que dichas respuestas se almacenen en la caché compartida.

Cache-Control: public

En general, cuando las paginas estan bajo Basic Auth o Digest Auth, el navegador enviará peticiones con la cabecera Authorization. Esto significa que la respuesta es de acceso-controlado para usuarios restringidos (quienes tienen cuenta), y esto es no compartidamente almacenado, incluso si tiene max-age.

Puedes usar la directiva public para desbloquear esa restricción.

Cache-Control: public, max-age=604800

Ten en cuenta que, s-maxage o must-revalidate también desbloquean esa restricción.

Si una petición no tiene la cabecera Authorization, o si ya estás usando s-maxage o must-revalidate en la respuesta, entonces no necesitas usar public.

must-understand

La directiva de respuesta must-understand indica que una caché debería de almacenar la respuesta solo si entiende los requisitos de almacenamiento basado en el codigo de estado.

must-understand debe ir emparejada con no-store, para un comportamiento como solución alternativa.

Cache-Control: must-understand, no-store

Si una caché no soporta must-understand, será ignorada. Si no-store está también presente, la respuesta no es almacenada.

Si una caché soporta must-understand, almacena la respuesta de acuerdo con los requisitos de cache basados en su código de estado.

no-transform

Algunos intermediarios transforman el contenido por diversas razones. Por ejemplo, algunos convierten las imagenes para reducir el tamaño de transferencia. En algunos casos, esto no es deseable para el proveedor de contenidos.

no-transform indica que cualquier intermediario (sin importar si implementa cache) no debería transformar los contenidos de la respuesta.

Nota: Web Light de Google es un intermediario de este tipo. Convirte las imagenes para minimizar los datos para almacenar en cache o para conexiones lentas, y soporta no-transform como una opción para evitar dicha función.

immutable

La directiva de respuesta immutable indica que la respuesta no será actualizada mientras sea reciente.

Cache-Control: public, max-age=604800, immutable

Una buena practica moderna para contenidos estáticos es incluir versión/hashes en sus URLs, mientras nunca se modifiquen los contenidos — pero en su lugar, cuando es necesario, actualizar las fuentes con nuevas versiones que tienen nuevos números de versión/hashes, de forma que las URLs son diferentes. Esto es conocido como el patrón cache-busting.

<script src=https://example.com/react.0.0.0.js></script>

Cuando un usuario recarga el navegador, el navegador le mandará una petición condicional para validar el servidor de origen. Pero no es necesario revalidar estos tipos de fuentes estáticas incluso cuando un usuario recarga el navegador, porque nunca son modificados. immutable indica a una cache que una respuesta es inmutable mientras es reciente, y evita ese tipo de peticiones condicionales innecesarias al servidor.

Cuando usas un patrón de cache-busting para fuentes y les aplicas un largo max-age, puedes también añadir immutable para evitar la revalidación.

stale-while-revalidate

La directiva de respuesta stale-while-revalidate indica que la cache puede reusar una respuesta antigua mientras se revalida en una caché.

Cache-Control: max-age=604800, stale-while-revalidate=86400

En el ejemplo anterior, la respuesta está actualizada durante 7 días (604800s). Después de 7 días se vuelve obsoleto, pero la caché puede reutilizarla para cualquier solicitud que se realice al día siguiente (86400s) — siempre que revalide la respuesta en segundo plano.

La revalidación hará que la memoria caché vuelva a estar actualizada, de modo que a los clientes les parezca que siempre estuvo actualizada durante ese período — Ocultando de forma efectiva la penalización por latencia de la revalidación.

Si no se produjo ninguna petición durante ese período, la caché se vuelve obsoleta y la próxima solicitud se revalidará normalmente.

stale-if-error

La directiva de respuesta stale-if-error indica que la memoria caché puede reutilizar una respuesta obsoleta cuando un servidor de origen responde con un error, o el error es generado localmente. Un error es cualquier respuesta con código de estado 500, 502, 503 o 504.

Cache-Control: max-age=604800, stale-if-error=86400

En el ejemplo anterior, la respuesta está actualizada durante 7 días (604800s). Después de 7 días, se vuelve obsoleta, pero se puede usar durante 1 día adicional (86400s) si el servidor responde con un error.

Después del período de tiempo de stale-if-error, la respuesta almacenada se vuelve obsoleta. Eso significa que el cliente recibirá una respuesta de error tal como el servidor de origen la envía.

Directivas de Peticiones

no-cache

La directiva de petición no-cache pide a las cachés que validen la respuesta con el servidor de origen antes de volver a usarla.

Cache-Control: no-cache

no-cache permite a los clientes solicitar la respuesta más actualizada incluso si la caché tiene una respuesta reciente.

Los navegadores generalmente agregan no-cache a las solicitudes cuando los usuarios fuerzan la recarga de una página.

no-store

La directiva de petición no-store permite a un cliente solicitar que las cachés se abstengan de almacenar la petición y la respuesta correspondiente, incluso si la respuesta del servidor de origen pudiera almacenarse.

Cache-Control: no-store

Tenga en cuenta que los principales navegadores no admiten peticiones con no-store.

max-age

La directiva de petición max-age=N indica que el cliente permite una respuesta almacenada que es generada en el servidor de origen dentro de N segundos, donde N puede ser cualquier número entero positivo (incluyendo el 0).

Cache-Control: max-age = 3600

En el caso anterior, si la respuesta con Cache-Control: max-age=604800 se almacenó en las cachés hace 3 horas, la caché no podría reutilizar esa respuesta.

Muchos navegadores usan esta directiva para recargar, como se explica a continuación.

Cache-Control: max-age=0

max-age=0 es una solución alternativa para no-cache, porque muchas implementaciones de caché antiguas (HTTP/1.0) no son compatibles con no-cache. Los navegadores más recientes siguen usando max-age=0 en "recargas" (por compatibilidad con versiones anteriores) y, alternativamente, usan no-cache para provocar una "recarga forzada".

Si el valor de max-age no es positivo (por ejemplo, -1) o no es entero (por ejemplo, 3599.99), el comportamiento del cache es indefinido. Sin embargo, la sección Calculating Freshness Lifetime de las especificaciones HTTP establece:

Caches are encouraged to consider responses that have invalid freshness information to be stale.

Traducción: Se recomienda a las caches considerar una respuesta como antigua si la información de edad es inválida.

En otras palabras, para cualquier valor de max-age que no es un entero positivo, el comportamiento de cache recomendado es de tratar el valor como si fuera 0.

max-stale

La directiva de solicitud max-stale=N indica que el cliente permite una respuesta almacenada que está obsoleta hasta N segundos.

Cache-Control: max-stale=3600

En el ejemplo anterior, si la respuesta con Cache-Control: max-age=604800 se almacenó en caché hace 3 horas, la caché no podría reutilizar esa respuesta.

Los clientes pueden usar este encabezado cuando el servidor de origen está inactivo o es demasiado lento y pueden aceptar respuestas almacenadas en caché incluso si son un poco antiguas.

Tenga en cuenta que los principales navegadores no admiten solicitudes con max-stale.

min-fresh

La directiva de petición min-fresh=N indica que el cliente permite una respuesta almacenada que está actualizada durante al menos N segundos.

Cache-Control: min-fresh=600

En el caso anterior, si la respuesta con Cache-Control: max-age=3600 se almacenó en las cachés hace 51 minutos, la caché no podría reutilizar esa respuesta.

Los clientes pueden usar este encabezado cuando el usuario requiere que la respuesta no solo sea actualizada, sino que también requiere que no se actualice durante un período de tiempo.

Tenga en cuenta que los principales navegadores no admiten peticiones con min-fresh.

no-transform

El mismo significado que no-transform tiene para una respuesta, pero para una petición en su lugar.

only-if-cached

El cliente indica que la caché debe obtener una respuesta ya almacenada en caché. Si una caché ha almacenado una respuesta, se reutiliza.

Casos de uso

Prevención del almacenamiento

Si no desea que una respuesta se almacene en las cachés, use la directiva no-store.

Cache-Control: no-store

Tenga en cuenta que no-cache significa "se puede almacenar pero no reutilizar antes de validar" — por lo que no es para evitar que se almacene una respuesta.

http example-bad Cache-Control: no-cache

En teoría, si las directivas están en conflicto, se debe respetar la directiva más restrictiva.
Así que el siguiente ejemplo básicamente no tiene sentido, porque private, no-cache, max-age=0 y

must-revalidate entran en conflicto con no-store.

http example-bad

conflicto Cache-Control: private, no-cache, no-store, max-age=0, must-revalidate

equivalete a Cache-Control: no-store

Almacenamiento en caché de activos estáticos con "cache busting"

Cuando crea activos estáticos con mecanismos de control de versiones/hashing, agregar una versión/hash al nombre de archivo o cadena de consulta es una buena manera de administrar el almacenamiento en caché.

Por ejemplo:

<!-- index.html --> <script src="/activos/react.min.js"></script>

La versión de la biblioteca de React cambiará cuando actualices la biblioteca, y hero.png también cambiará cuando edites la imagen. Por lo tanto, son difíciles de almacenar en un caché con max-age.

En tal caso, podría abordar las necesidades de almacenamiento en caché utilizando una versión numerada específica de la biblioteca e incluyendo el hash de la imagen en su URL.

<!-- index.html --> <script src="/assets/react.0.0.0min.js"></script>

Puede agregar un valor grande de max-age e immutable, porque el contenido nunca cambiará.

/assets/* Cache-Control: max-age=31536000, immutable

Cuando actualiza la biblioteca o edita la imagen, el nuevo contenido debe tener una nueva URL y los cachés no se reutilizan. Eso se llama el patrón "cache busting".

Utilice no-cache para asegurarse de que la respuesta HTML en sí misma no se almacene en caché. no-cache puede causar la revalidación y el cliente recibirá correctamente una nueva versión de la respuesta HTML y los activos estáticos.

/index.html Cache-Control: no-cache

Nota: si index.html se accede mediante Basic Authentication o Digest Authentication, los archivos bajo /assets no se almacenarán en la memoria caché compartida. Si los archivos /assets/ son adecuados para almacenar en un caché compartido, también necesita uno de public, s-maxage o must-revalidate.

Contenidos siempre actualizados

Para los contenidos que se generan dinámicamente, o que son estáticos pero se actualizan con frecuencia, deseas que un usuario reciba siempre la versión más actualizada.

Si no agrega un encabezado Cache-Control porque la respuesta no está destinada para almacenarse en caché, podría causar un resultado inesperado. El almacenamiento en caché puede almacenarlo

heurísticamente — por lo que si tienes algún requisito para el almacenamiento en caché, siempre debes indicarlo explícitamente en el encabezado Cache-Control.

Agregar no-cache a la respuesta provoca la revalidación en el servidor, por lo que puede entregar una respuesta nueva cada vez — o si el cliente ya tiene una nueva, simplemente responda 304 Not Modified.

Cache-Control: no-cache

La mayoría de las cachés HTTP/1.0 no son compatibles con las directivas no-cache, por lo que históricamente se usaba max-age=0 como solución alternativa. Pero usar solo max-age=0 podría hacer que se reutilice una respuesta obsoleta cuando los cachés se desconecten del servidor de origen. must-revalidate aborda eso. Es por eso que el siguiente ejemplo es equivalente a no-cache.

Cache-Control: max-age=0, must-revalidate

Pero hoy en día, puedes simplemente usar no-cache en su lugar.

Borrar un caché ya almacenado

Desafortunadamente, no hay directivas de caché para borrar las respuestas ya almacenadas de las cachés.

Imagine que los clientes/cachés almacenan una respuesta nueva para una ruta, sin solicitud de vuelo al servidor. No hay nada que un servidor pueda hacer en esa ruta.

Alternativamente, Clear-Site-Data puede borrar la memoria caché del navegador para un sitio. Pero tenga cuidado: eso borra todas las respuestas almacenadas para un sitio — y solo en los navegadores, no para un caché compartido.

Especificaciones

Compatibilidad en navegadores

Véase también

- Caché en HTTP
- Tutorial de almacenamiento en caché para autores web y webmasters
- Prácticas recomendadas de almacenamiento en caché y trucos de edad máxima
- Control de caché para civiles
- RFC 9111 – Caché en HTTP
- RFC 5861 – Extensiones de control de caché HTTP para contenido obsoleto
- RFC 8246 – Respuestas HTTP inmutables

Strict-Transport-Security

HTTP Strict-Transport-Security (a menudo abreviado como HSTS) es una característica de seguridad que permite a un sitio web indicar a los navegadores que sólo se debe comunicar con HTTPS en lugar de usar HTTP.

| Tipo de Encabezado | Encabezado de Respuesta |

| Nombre de Encabezado Prohibido | no |

Sintaxis

Strict-Transport-Security: max-age=<expire-time> Strict-Transport-Security: max-age=<expire-time>; includeSubDomains Strict-Transport-Security: max-age=<expire-time>; preload

Directivas

- max-age=<expire-time>
Es el tiempo, en segundos, que el navegador debe recordar que el sitio solo debe ser accsible usando HTTPS.
- includeSubDomains
Si este parámetro opcional está especificado, la regla aplica para todos los subdominios del sitio.
- preload
Ver Precargando Strict Transport Security para mas detalles. No es parte de la especificación.

Descripción

Si un sitio web acepta una conexión a través de HTTP y redirecciona a HTTPS, el usuario en este caso podría inicialmente hablar a la versión no encriptada del sitio antes de ser redireccionado, si por ejemplo el usuario tipea http://www.foo.com/ o incluso solo foo.com.

Esto habilita el potencial ataque man-in-the-middle, donde el redireccionamiento podría ser aprovechado para enviar al usuario a un sitio malicioso en lugar de la versión segura de la página original.

El encabezado HTTP Strict Transport Security permite a un sitio web informar al navegador que nunca cargue el sitio usando HTTP y que debe automáticamente convertir todos los intentos de acceso HTTP a HTTPS.

NOTE: El encabezado Strict-Transport-Security es ignorado por el navegador cuando el sitio es accedido usando HTTP; esto es porque un atacante podría interceptar las conexiones HTTP e inyectar el encabezado o removerlo. Cuando el sitio es accedido a través de HTTPS con un certificado sin errores, el navegador sabe que el sitio es capaz de usar HTTPS y cumple con lo indicado en el encabezado Strict-Transport-Security.

Un escenario de ejemplo

Tu ingresas a una red WiFi libre en un areopuerto y empiezas a nevegar por el internet visitando tu servicio de banca en linea para revisar tu estado de cuenta y pagar algunas cuentas. Desafortunadamente, el punto de acceso que estás usando es actualmente un computador portátil de un hacker. Ellos están interceptando todas tus solicitudes originales HTTP y redireccionando a un clone del sitio de tu banco en lugar del sitio real. Ahora tus datos privados están expuestos al hacker.

Strict Transport Security resuelve este problema; siempre que hayas ingresado al sitio de tu banco una vez usando HTTPS y el sitio del banco use Strict Transport Security. Tu navegador sabe que debe usar HTTPS, lo cual previene el hacker realizar este tipo de ataque.

Referencia HTTP de MDN: códigos de estado y encabezados (selección)

Como el navegador lo maneja

La primera vez que accediste al sitio usando HTTPS y este retornó el encabezado Strict-Transport-Security, el navegador registra esta información, de tal manera que en futuros intentos para cargar el sitio usando HTTP va a usar en su lugar HTTPS automáticamente.

Cuando el tiempo de expiración especificado por el encabezado Strict-Transport-Security haya pasado, el siguiente intento de cargar el sitio a través de HTTP se va a procesar de forma normal.

En cualquier momento que el encabezado Strict-Transport-Security sea entregado el navegador, este actualiza el tiempo de expiración para el sitio, así los sitios pueden refrescar su información y prevenir el tiempo de expiración. Para deshabilitarlo sería necesario configurar max-age a 0 sobre una conexión HTTPS, entonces automáticamente expira el encabezado Strict-Transport-Security permitiendo acceso vía HTTP.

Precargando Strict Transport Security

Google mantiene un servicio de precarga HSTS. Siguiendo la siguiente guía y enviando un dominio válido, los navegadores nunca se conectarán a utu dominio usando una conexión insegura. Mientras el servicio esté sobre Google, todos los navegadores tienen determinado intentar usar la lista precargada.

- Información de lista precargada HSTS en Chrome: <https://www.chromium.org/hsts>
- Consulta de lista precargada de Firefox: nsSTSPreloadList.inc

Ejemplos

Todos los presentes y futuros subdominios usarán HTTPS durante 1 año.

This blocks access to pages or sub domains that can only be served over HTTP.

Strict-Transport-Security: max-age=31536000; includeSubDomains

Especificaciones

Compatibilidad de navegadores

Ver también

- Blog post: HTTP Strict Transport Security has landed!
- Blog post: HTTP Strict Transport Security (force HTTPS)
- OWASP Article: HTTP Strict Transport Security
- Wikipedia: HTTP Strict Transport Security

Content-Security-Policy

La cabecera HTTP Content-Security-Policy en la respuesta permite a los administradores de un sitio web controlar los recursos que el User-Agent puede cargar a una pagina. Con algunas (Poquísimas) excepciones, las políticas implican principalmente especificar el servidor de origen la protección de puntos finales del script. Esto ayuda a protegerse contra ataques Cross-site scripting (XSS).

Para mas información, ve también este articulo en Content Security Policy (CSP).

<tbody>

Tipo de cabecera |
Response header |

Forbidden request header |
no |

</tbody>

Sintaxis

Content-Security-Policy: <policy-directive>; <policy-directive>

Directivas

Fetch directive

"Fetch directives" controla la ubicación o ubicaciones desde la cual se pueden cargar ciertos tipos de recursos

Lista de Content Security Policy Fetch directives

- child-src
Define los origenes válidos para web workers y contextos de navegación anidados cargados usando elementos como and .

> [!WARNING]
> En lugar de child-src, los autores quienes deseen regular los contextos de navegación anidados y "workers" deberían usar las directivas frame-src y worker-src, respectivamente.

- connect-src
Restringe las URLs que pueden ser cargados usando scripts.
- default-src
Serves as a fallback for the other Fetch directive.
- font-src
Especifica origenes válidos para las fuentes cargadas usando .
- frame-src
Especifica origenes válidos para contextos de navageción anidada cargados usando elementos como y .
- img-src
Especifica origenes válidos de imágenes y favicons.
- manifest-src
Especifica origenes válidos de archivos de manifiesto de una aplicación.
- media-src
Especifica origenes válidos para carga de archivos usando elementos como , y .
- object-src
Specifies valid sources for the and elements.

Elements controlled by object-src are perhaps coincidentally considered legacy HTML elements and aren't recieving new standardized features (such as the security attributes sandbox or allow for <iframe>). Therefore it is recommended to restrict this fetch-directive (e.g. explicitly set object-src 'none' if possible).

- prefetch-src
Specifies valid sources to be prefetched or prerendered.
- script-src
Specifies valid sources for JavaScript.
- style-src
Specifies valid sources for stylesheets.
- webrtc-src
Specifies valid sources for WebRTC connections.
- worker-src
Specifies valid sources for Worker, SharedWorker, or ServiceWorker scripts.

Document directive

Document directives govern the properties of a document or worker environment to which a policy applies.

- base-uri
Restricts the URLs which can be used in a document's element.
- plugin-types
Restricts the set of plugins that can be embedded into a document by limiting the types of resources which can be loaded.
- sandbox
Enables a sandbox for the requested resource similar to the sandbox attribute.
- disown-opener
Ensures a resource will disown its opener when navigated to.

Navigation directive

Navigation directives govern to which location a user can navigate to or submit a form to, for example.

- form-action
Restricts the URLs which can be used as the target of a form submissions from a given context.
- frame-ancestors
Specifies valid parents that may embed a page using , , , or .
- navigate-to
Restricts the URLs to which a document can navigate by any means (a, form, window\.location, window\.open, etc.)

Reporting directive

Reporting directives control the reporting process of CSP violations. See also the Content-Security-Policy-Report-Only header.

- report-uri
Instructs the user agent to report attempts to violate the Content Security Policy. These violation reports consist of JSON documents sent via an HTTP POST request to the specified

URI.

> [!WARNING]
> Though the report-to directive is intended to replace the deprecated report-uri directive, report-to isn't supported in most browsers yet. So for compatibility with current browsers while also adding forward compatibility when browsers get report-to support, you can specify both report-uri and report-to:
>
>
> Content-Security-Policy: ...; report-uri https://endpoint.example.com; report-to groupname
>
>
> In browsers that support report-to, the report-uri directive will be ignored.

- report-to
Fires a SecurityPolicyViolationEvent.

Other directives

- block-all-mixed-content
Prevents loading any assets using HTTP when the page is loaded using HTTPS.
- referrer
Used to specify information in the referer (sic) header for links away from a page. Use the Referrer-Policy header instead.
- require-sri-for
Requires the use of SRI for scripts or styles on the page.

<!---->

- trusted-types
Used to specify a whitelist of Trusted Types policies (Trusted Types allows applications to lock down DOM XSS injection sinks to only accept non-spoofable, typed values in place of strings).

<!---->

- upgrade-insecure-requests
Instructs user agents to treat all of a site's insecure URLs (those served over HTTP) as though they have been replaced with secure URLs (those served over HTTPS). This directive is intended for web sites with large numbers of insecure legacy URLs that need to be rewritten.

CSP in workers

Workers en general no se rigen por las politicas de seguridad de contenido de el documento (o padre del worker) que los creó. To specify a content security policy for the worker, set a Content-Security-Policy response header for the request which requested the worker script itself.

The exception to this is if the worker script's origin is a globally unique identifier (for example, if its URL has a scheme of data or blob). In this case, the worker does inherit the content security policy of the document or worker that created it.

Multiple content security policies

CSP allows multiple policies being specified for a resource, including via the Content-Security-Policy header, the Content-Security-Policy-Report-Only header and a element.

You can use the Content-Security-Policy header more than once like in the example below. Pay special attention to the connect-src directive here. Even though the second policy would allow the connection, the first policy contains connect-src 'none'. Adding additional policies can only further restrict the capabilities of the protected resource, which means that there will be no connection allowed and, as the strictest policy, connect-src 'none' is enforced.

Content-Security-Policy: default-src 'self' http://example.com;
connect-src 'none';
Content-Security-Policy: connect-src http://example.com/;
script-src http://example.com/

Ejemplos

Example: Disable unsafe inline/eval, only allow loading of resources (images, fonts, scripts, etc.) over https:

// header Content-Security-Policy: default-src https:

// meta tag <meta http-equiv="Content-Security-Policy" content="default-src https:">

Example: Pre-existing site that uses too much inline code to fix but wants to ensure resources are loaded only over https and disable plugins:

Content-Security-Policy: default-src https: 'unsafe-eval' 'unsafe-inline'; object-src 'none'

Example: Don't implement the above policy yet; instead just report violations that would have occurred:

Content-Security-Policy-Report-Only: default-src https:; report-uri /csp-violation-report-endpoint/

See Mozilla Web Security Guidelines for more examples.

Especificaciones

Compatibilidad con navegadores

Mirar tambien

- Content-Security-Policy-Report-Only
- Learn about: Content Security Policy
- Content Security in WebExtensions
- Display security and privacy policies In Firefox Developer Tools
- Adopting a strict policy

Set-Cookie

La cabecera de respuesta HTTP Set-Cookie se usa para enviar cookies desde el servidor al agente

de usuario, así el agente de usuario puede enviarlos de vuelta al servidor.Para más información, visite la guía para cookies HTTP.

<tbody>

Tipo de cabecera |

Response header
|

Forbidden header name
|
no |

</tbody>

Sintaxis

Set-Cookie: <cookie-name>=<cookie-value> Set-Cookie: <cookie-name>=<cookie-value>; Expires=<date> Set-Cookie: <cookie-name>=<cookie-value>; Max-Age=<non-zero-digit> Set-Cookie: <cookie-name>=<cookie-value>; Domain=<domain-value> Set-Cookie: <cookie-name>=<cookie-value>; Path=<path-value> Set-Cookie: <cookie-name>=<cookie-value>; Secure Set-Cookie: <cookie-name>=<cookie-value>; HttpOnly

Set-Cookie: <cookie-name>=<cookie-value>; SameSite=Strict Set-Cookie: <cookie-name>=<cookie-value>; SameSite=Lax Set-Cookie: <cookie-name>=<cookie-value>; SameSite=None

// Es posible usar múltiples directivas, por ejemplo: Set-Cookie: <cookie-name>=<cookie-value>; Domain=<domain-value>; Secure; HttpOnly

Directivas

- Si se omite, se define el húesped como la URL del documento actual, sin incluir subdominios.
- Contrario a la especificación anterior, puntos principales en el nombre del domino (.ejemplo.com) son ignorados.
- Si un dominio es especificado, los subdominios son también incluidos.

<!---->

- If omitted, defaults to the host of the current document URL, not including subdomains.
- Contrary to earlier specifications, leading dots in domain names (.example.com) are ignored.
- If a domain is specified, subdomains are always included.

<!---->

- <cookie-name>=<cookie-value>
Una cookie comienza con un par nombre-valor:
- Un <cookie-name> puede ser cualquier cosa excepto caracteres de control (CTLs) o espacios y tabulaciones. Tampoco debe contener caracteres de separación como los siguientes: ( ) < > @ , ; : \ " / [ ] ? = { }.
- Un <cookie-value> opcionalmente puede ser establecido dentro de comillas dobles y se permite usar cualquier caracter US-ASCII excluyendo CTLs, espacios en blanco, comillas dobles, comas,

punto y coma y la barra invertida. Codificación: Muchas implementaciones realizan codificación de URL sobre los valores de la cookie, aunque esto no es requerido por la especificación RFC. Esto ayuda a satisfacer los requerimientos sobre los caracteres permitidos para \<cookie-value>.
- Prefijo __Secure-: Las cookies cuyo nombre comience por __Secure- (los guiones forman parte del prefijo) deben ser establecidas con la bandera secure y deben provenir de una página segura (HTTPS).
- Prefijo __Host-: Las cookies cuyo nombre comience por __Host- deben ser establecidas con la bandera secure, provenir de una página segura (HTTPS), no deben tener especificado un dominio (y por tanto no son enviadas a subdominios) y la ruta debe ser "/".

- Expires=\<date>
El tiempo de vida máximo útil de una cookie como marca HTTP-date timestamp. Ver Date para el detalle del formato.

Si no está especificado, la cookie tendrá el tiempo de vida de una session cookie. Una sesión finaliza cuando el cliente lo culmina, esto quiere decir que la sesión será removida en ese punto.

> [!WARNING]
> Sin embargo, muchos navegadores web tiene una caracteristica llamada "restaurar" que almacenará todas tus pestañas para tenerlas lista cuando el navegador sea usado nuevamente. Las cookies de sesión tambien estarán presentes como si nunca se hubiera cerrado el navegador.

Cuando una fecha de Expires es definida, la fecha límite es relativa al cliente donde la cookie se define, no en el servidor.

- Max-Age=<non-zero-digit>
Número de segundos hasta que la cookie expire. Un cero o un número negativo hace expirar la cookie inmediatamente. Los navegadores antiguos (ie6, ie7, and ie8) no soportan max-age. Para otros navegadores, si ambos estan definidos (Expires y Max-Age), Max-Age tendrá mayor importancia.
- Domain=<domain-value>
Anfitriones donde la cookie será enviada.
- Si se omite, por defecto el huésped es la URL del documento actual, sin incluir subdominios.
- Contraria a anteriores especificaciones, los puntos principales en nombres de dominio (.example.com) son ignorados.
- Si un dominio es especificado, los subdominios son siempre incluídos.

- Path=<path-value>
Una ruta que debe existir en la URL solicitada, o el navegador no enviará el encabezado Cookie.
El caracter diagonal (/) es interpretado como un separador de directorios y subdirectorios que serán también comparados: para Path=/docs, /docs, /docs/Web/, y /docs/Web/HTTP todos tendrán que coincidir.
- Secure
Una cookie segura es sólo enviada al servidor cuando una petición es enviada con el esquema https:. (Sin embargo, información confidencial nunca debería ser almacenada en Cookies HTTP, como todo el mecanismo es However, confidential information should never be stored in HTTP Cookies, ya que todo el mecanismo es inherentemente inseguro y no encripta ninguna información.)

> [!NOTE]

> Sitios inseguros (http:) no puenden almacenar directivas "seguras" apartir de Chrome 52+ y Firefox 52+.

- HttpOnly
Impide que el código JavaScript acceda a la cookie. Por ejemplo, a través de la propiedad Document.cookie, y la API XMLHttpRequest, o la API Request. Esto mitiga los ataques contra scripts cross-site (XSS).
- SameSite=<samesite-value>
- Strict
- Lax
- None

Afirma que una cookie no debe ser enviada con peticiones cruzadas, proveiendo alguna protección contra ataques de falsificación de solicitudes cruzadas (CSRF).

> [!NOTE]
> Los navegadores haciendo migraciones para que el comportamiento por defecto de las cookies en lugar de default sea SameSite=Lax. Si una cookie necesita ser enviada en peticiones cruzadas, se tiene que optar por no restringir el mismo sitio (SameSite) usando la directiva None. La directiva None requiere el atributo Secure.

Ejemplos

Session cookie

Las cookies de sesión son removidas cuando el cliente se apaga. Las cookies son cookies de sesión si no se especifican las directivas Expires o Max-Age.

Set-Cookie: sessionId=38afes7a8

Permanent cookie

En lugar de expirar cuando el cliente se cierra, las cookies permanentes expiran en una fecha especifica (Expires) o después de una longitud de tiempo determinado (Max-Age).

Set-Cookie: id=a3fWa; Expires=Wed, 21 Oct 2015 07:28:00 GMT

Set-Cookie: id=a3fWa; Max-Age=2592000

Invalid domains

Una cookie para un dominio que no incluya el servidor que la defina debería ser rechazada por el agente de usuario.

La siguiente cookie sera rechazada si se define por el servidor ubicado en originalcompany.com como:

Set-Cookie: qwerty=219ffwef9w0f; Domain=somecompany.co.uk

Una cookie para el subdomino del dominio actual será rechazada.

La siguiente cookie sera rechazada si el servidor anfitrión en example.com la define como:

Set-Cookie: sessionId=e8bb43229de9; Domain=foo.example.com

Cookie prefixes

Los nombres de las cookies prefijadas con __Secure- o __Host- pueden ser solo usadas si son definidas con la directiva secure desde un origen (HTTPS) seguro.

Además, cookies con el prefijo __Host- deben tener una ruta de / (significando cualquier ruta del huésped) y no debe tener un atributo Domain.

WARNING: Para clientes que no implementan prefijos para las cookies, no se puede contar con que estas garantías adicionales, y las cookies prefijadas sean aceptadas.

// Ambas aceptadas al venir de un origen seguro (HTTPS) Set-Cookie: __Secure-ID=123; Secure; Domain=example.com Set-Cookie: __Host-ID=123; Secure; Path=/

// Rechazada por faltar la directiva Secure Set-Cookie: __Secure-id=1

// Rechazada por faltar la directiva Path=/ Set-Cookie: __Host-id=1; Secure

// Rechazada por definir un dominio Set-Cookie: __Host-id=1; Secure; Path=/; domain=example.com

Especificaciones

Compatibilidad con navegadores

Notas de compatibilidad

- Empezando con Chrome 52 y Firefox 52, sitios inseguros (http:) no pueden definirse cookies con la directiva 'secure'.

Ver también

- HTTP cookies
- Cookie
- Document.cookie

X-Frame-Options

El encabezado de respuesta HTTP X-Frame-Options puede ser usado para indicar si debería permitírsele a un navegador renderizar una página en un , , u . Las páginas web pueden usarlo para evitar ataques de click-jacking, asegurándose de que su contenido no es embebido en otros sitios.

La seguridad añadida sólo es proporcionada si el usuario que está accediendo al documento está utilizando un navegador que soporte X-Frame-Options.

NOTE: El encabezado HTTP Content-Security-Policy tiene una directiva Content-Security-Policy/frame-ancestors que deja obsoleto este encabezado para los navegadores compatibles.

<tbody>

Tipo de encabezado |
Response header |

Forbidden request header |
no |

</tbody>

Sintaxis

Existen dos posibles directivas para X-Frame-Options:

X-Frame-Options: DENY X-Frame-Options: SAMEORIGIN

Directivas

Si especifica DENY, fallarán no sólo los intentos de cargar la página en un marco desde otros sitios, sino que fallarán cuando sea cargada desde el mismo sitio. Por otro lado, si especifica SAMEORIGIN, puede usar la página en un marco mientras el sitio que la incluya sea el mismo que la sirve.

- DENY
La página no puede ser mostrada en un marco, independiente del sitio que esté intentándolo.
- SAMEORIGIN
La página sólo puede ser mostrada en un marco del mismo origen que dicha página. La especificación deja que los proveedores de navegadores decidan si esta opción se aplica al nivel superior, al padre o a toda la cadena, aunque se argumenta que la opción no es muy útil a menos que todos los padres también estén en el mismo origen (ver Error 725490 en Firefox). Consulte también Browser compatibility para obtener detalles de soporte.
- ALLOW-FROM uri
Esta es una directiva obsoleta que ya no funciona en navegadores modernos. No usar. En el soporte de navegadores antiguos, una página se puede mostrar en un marco solo en el origen especificado uri. Tenga en cuenta que en versiones anteriores de Firefox esto todavía sufre del mismo problema que SAMEORIGIN — no verifica los antecesores del marco para ver si están en el mismo origen. El encabezado HTTP Content-Security-Policy tiene una directiva Content-Security-Policy/frame-ancestors que puede usar en su lugar.

Ejemplos

NOTE: ¡Configurar X-Frame-Options en el tag es inútil! Por ejemplo, <meta http-equiv="X-Frame-Options" content="deny"> no tiene efecto. ¡No lo use! X-Frame-Options sólo funcionará configurandolo a tráves del encabezado HTTP, como en los ejemplos a continuación.

Configurando Apache

Agregue lo siguiente a la configuración de su sitio para que Apache envíe el encabezado X-Frame-Options para todas las páginas:

Header always set X-Frame-Options "SAMEORIGIN"

Para que Apache envíe X-Frame-Options deny, agregue lo siguiente a la configuración de su sitio:

Header set X-Frame-Options "DENY"

Configurando nginx

Para configurar nginx a que envíe el encabezado X-Frame-Options, agregue esto a la configuración, ya sea http, server o location:

add_header X-Frame-Options SAMEORIGIN always;

Configurando IIS

Para hacer que IIS envíe el encabezado X-Frame-Options, agrege esto al archivo Web.config de su sitio:

<system.webServer> ...

<httpProtocol>
<customHeaders>
<add name="X-Frame-Options" value="SAMEORIGIN" />
</customHeaders>
</httpProtocol>

... </system.webServer>

O consulte este artículo de soporte de Microsoft sobre cómo establecer esta configuración mediante la interfaz de usuario del Administrador de IIS

Configurando HAProxy

Para hacer que HAProxy envíe el encabezado X-Frame-Options, agrege lo siguiente a su configuración front-end, listen, o backend:

rspadd X-Frame-Options:\ SAMEORIGIN

Opcionalmente, en nuevas versiones:

http-response set-header X-Frame-Options SAMEORIGIN

Configurando Express

Para hacer que Express envíe el encabezado X-Frame-Options, puede usar helmet que utiliza frameguard para establecer el encabezado. Agregue lo siguiente a la configuración de su servidor:

const helmet = require("helmet"); const app = express(); app.use(helmet.frameguard({ action: "SAMEORIGIN" }));

Opcionalmente, puede usar frameguard directamente:

const frameguard = require("frameguard"); app.use(frameguard({ action: "SAMEORIGIN" }));

Referencia HTTP de MDN: códigos de estado y encabezados (selección)

Especificaciones

Compatibilidad con navegadores

Vea también

- Content-Security-Policy directiva Content-Security-Policy/frame-ancestors
- HTTP Header Field X-Frame-Options - RFC 7034
- ClickJacking Defenses - IEBlog
- Combating ClickJacking with X-Frame-Options - IEInternals

ETag

El encabezado de respuesta de HTTP ETag es un identificador para una versión específica de un recurso. Permite a la memoria caché ser más eficiente, y ahorrar ancho de banda, en tanto que un servidor web no necesita enviar una respuesta completa si el contenido no ha cambiado. Por otro lado, si el contenido cambió, los etags son útiles para ayudar a prevenir actualizaciones simultáneas de un recurso de sobre-escribirlo por otro ("colisiones en el aire").

Si el recurso en una URL dada cambia, un valor Etag debe ser generado. De esta forma los Etags son muy similares a las huellas digitales y pueden también usarse para propósitos de rastreo por algunos servidores. Un comparativo de ellos permite rápidamente determinar cuándo dos representaciones de un recurso son las mismas, pero podrían también configurarse para persistir indefinidamente por un servidor en rastreo.

| Tipo de Encabezado | Response header |

| Forbidden request header | no |

Sintaxis

ETag: W/"<valor_etag>" ETag: "<valor_etag>"

Directivas

- W/
'W/' (sensible a mayúsculas) indica que se usa un validador débil. Los validadores débiles son fáciles de generar pero son menos útiles para realizar comparativos. Los validadores fuertes son ideales para realizar comparativos pero pueden ser muy difíciles de generar de forma eficiente. Los valores Etag débiles de dos representaciones de los mismos recursos podrían ser semánticamente equivalentes, pero no idénticos byte por byte. Esto significa que los Etag débiles previenen el almacenamiento en caché cuando el range request por byte es usado, a su vez los Etag fuertes permiten que los range request puedan utilizar el almacenamiendo en caché.
- "\<valor_etag>"
Las Etiquetas de Entidad (ETags) representan de forma única a los recursos. Son una cadena de caracteres ASCII puestas entre comillas dobles (Como "675af34563dc-tr34"). El método por el cual se generan los valores ETag no está especificado. Muchas veces, se usa una etiqueda del contenido, una etiqueta de la fecha y hora de la última modificación, o sólo una revisión. Por ejemplo, MDN usa una etiqueda de dígitos hexadecimales para el contenido wiki.

Ejemplos

ETag: "33a64df551425fcc55e4d42a148795d9f25f89d4" ETag: W/"0815"

Evitando las colisiones en el aire

Con la ayuda del ETag y los encabezados If-Match se puede ser capaz de detectar las colisiones de edición en el aire.

Por ejemplo cuando se edita MDN, el contenido wiki actual es etiquetado y puesto en un Etag en la respuesta:

ETag: "33a64df551425fcc55e4d42a148795d9f25f89d4"

Cuando se guarda los cambios de una página a una página wiki (datos posteados), la petición POST contendrá el encabezado que contiene los valores ETag para revisar la frescura entre ellas.

If-Match: "33a64df551425fcc55e4d42a148795d9f25f89d4"

Si las etiquetas no concuerdan, significa que el documento ha sido editado de por sí y se lanza un error 412 Precondition Failed.

Caching de los recursos invariados

Otro caso típico del uso del encabezado ETag es el cacheo de recursos que no han variado. Si un usuario visita una URL dada nuevamente (la que tiene un conjunto ETag), y está viciado, es decir que es muy viejo para considerarlo usable, el cliente enviará el valor de su ETag a través de un campo de encabezado If-None-Match:

If-None-Match: "33a64df551425fcc55e4d42a148795d9f25f89d4"

El servidor compara el ETag del cliente (enviado con un If-None-Match) con el ETag para su versión actual del recurso y si ambos valores concuerdan (esto es, el recurso no ha cambiado), el servidor envió un estado 304 Not Modified, sin ningún cuerpo, lo cual le dice al cliente que la versión cacheada de la respuesta todavía es buena para usar (refrescar).

Especificaciones

Compatibilidad con navegadores

Vea también

- If-Match
- If-None-Match
- 304 Not Modified
- 412 Precondition Failed
- W3C Note: Editing the Web – Detecting the Lost Update Problem Using Unreserved Checkout

Content-Disposition

En una respuesta HTTP regular, el encabezado Content-Disposition indica si el contenido se espera que se muestre en línea en el navegador, esto es, como una o como parte de una página web, o como un archivo adjunto, que se puede descargar y guardar localmente.

En un cuerpo multipart/form-data, el encabezado general Content-Disposition puede ser utilizado en la subparte de un cuerpo multipartes para dar información acerca del campo al que aplica. La subparte es delimitada por el boundary (límite) definido en el encabezado Content-Type. Cuando se utiliza en el propio cuerpo, Content-Disposition no tiene efecto.

El encabezado Content-Disposition está definido en el contexto de mensajes MIME para correos electrónicos, pero sólo un subconjuto de los parámetros posibles aplican a formularios HTTP y peticiones POST. Sólo el valor form-data, como las directivas opcionales name and filename, pueden ser utilizadas en el contexto HTTP.

| Tipo de encabezado | Response header (para el cuerpo principal) General header (para una subparte de un cuerpo multipartes) |

| Forbidden request header | no |

Sintaxis

Como encabezado de respuesta para cuerpo principal

El primer parámetro en el contexto HTTP o es inline (valor predeterminado, indicando que puede ser mostrado dentro de una página web, o como la página web) o attachment (indicando que será descargado; la mayoría de los navegadores mostrando un diálogo 'Guardar como', prellenado con el valor del parámetro filename, en caso de estar presente).

Content-Disposition: inline Content-Disposition: attachment Content-Disposition: attachment; filename="filename.jpg"

Como encabezado para un cuerpo multipartes

El primer parámetro en el contexto HTTP siempre es form-data; parámetros adicionales son insensibles a mayúsculas y tienen argumentos que usan sintaxis de textos entre comillas después del signo de '='. Múltiples parámetros se separan por punto y coma (';').

Content-Disposition: form-data Content-Disposition: form-data; name="fieldName" Content-Disposition: form-data; name="fieldName"; filename="filename.jpg"

Directivas

- name
Es seguida de un texto que contiene el nombre del campo de HTML en el formulario a la que el contenido de la subparte refiere. Cuando se usan múltiples archivos en el mismo campo (por ejemplo, el atributo multiple de un elemento), puede haber varias subpartes con el mismo nombre.
Un name con valor de 'charset' indica que la parte no es un campo HTML, sino el conjunto de caracteres predeterminado para partes sin información explícita sobre su conjunto de caracteres.
- filename
Es seguida de un texto que contiene el nombre original del archivo transmitido. Siempre es opcional y no debe ser utilizado a ciegas por la aplicación: información sobre la ruta debe ser despojada, y se debe realizar una conversión a las reglas del sistema de archivos del

servidor. Este parámetro provee mayormente información indicativa. Cuando se usa en combinación con Content-Disposition: attachment, es utilizado como el nombre de archivo predeterminado en caso de que se presente al usuario un diálogo de 'Guardar como'.
- filename*

Los parámetros filename y filename* difieren únicamente en que filename* utiliza encodeado definido en RFC 5987. Cuando ambos están presentes en un valor de campo de encabezado simple, filename* es preferido sobre filename cuando ambos están presentes y entendidos.

Ejemplos

Una respuesta generando el diálogo 'Guardar como':

200 OK Content-Type: text/html; charset=utf-8 Content-Disposition: attachment; filename="genial.html" Content-Length: 22

<HTML>Guárdame!</HTML>

Este archivo simple de HTML será guardado como una descarga regular en lugar de mostrarse en el navegador. La mayoría de los navegadores propondrá guardarlo como genial.html ya que es el nombre (predeterminado).

Un ejemplo de un formulario HTML, publicado usando el formato multipart/form-data que hace uso del encabezado Content-Disposition:

POST /test.html HTTP/1.1 Host: example.org Content-Type: multipart/form-data;boundary="boundary"

--boundary
Content-Disposition: form-data; name="campo1"

valor1
--boundary
Content-Disposition: form-data; name="campo2"; filename="ejemplo.txt"

valor2
--boundary--

Especificaciones

Compatibilidad con navegadores

Notas de compatibilidad

- Firefox 5 maneja el encabezado de respuesta HTTP Content-Disposition más efectivamente si ambos parámetros filename y filename* están presentes; observa todos los nombres presentes, usando el parámetro filename* si uno está disponible, incluso si el parámetro filename está incluido antes. Previamente, el primer parámetro en encontrarse sería usado, de este modo se previene el uso de un nombre más apropiado. Mira Error 588781 en Firefox.

Ver también

- Formularios HTML
- El Content-Type definiendo el límite de un cuerpo multipartes.

- La interfaz FormData usada para manipular datos de formulario para uso en la API XMLHttpRequest.

X-Content-Type-Options

El encabezado HTTP de respuesta X-Content-Type-Options es un marcador utilizado por el servidor para indicar que los tipos MIME anunciados en los encabezados Content-Type no se deben cambiar ni seguir. Esto permite desactivar el MIME type sniffing, o, en otras palabras, es una manera de decir que los webmasters sabían lo que estaban haciendo.

Este encabezado fue introducido por Microsoft en IE 8 para que los webmasters bloquearan el rastreo de contenido, pudiendo transformar tipos MIME no ejecutables en tipos MIME ejecutables. Desde entonces, otros navegadores lo han introducido, incluso con algoritmos de detección MIME menos agresivos.

Los evaluadores de seguridad del sitio suelen esperar que este encabezado aparezca.

NOTE: nosniff solo se aplican a los tipos "script" y "style". Además la aplicación de nosniff a las imágenes resulto ser incompatible con los sitios web existentes.

| Header type | Response header |

| Forbidden request header | no |

Sintaxis

X-Content-Type-Options: nosniff

Directivas

- nosniff
Bloquea una solicitud si el tipo solicitado es
- "style" y el tipo MIME no es "text/css", o
- "script" y el tipo MIME no es un JavaScript MIME type.

Especificaciones

Compatibilidad con navegadores

Vea también

- Content-Type
- The original definition of X-Content-Type-Options by Microsoft.
- The Mozilla Observatory tool testing the configuration (including this header) of Web sites for safety and security
- Mitigating MIME Confusion Attacks in Firefox

Referrer-Policy

La cabecera Referrer-Policy de HTTP determina qué datos de referente, de entre los que se envían con la cabecera Referer, deben incluirse con las solicitudes realizadas.

Referencia HTTP de MDN: códigos de estado y encabezados (selección)

<tbody>

Tipo de cabecera |
Response header |

Forbidden request header |
no |

</tbody>

Sintaxis

Observe que Referer es una falta de ortografía; en inglés, la palabra correcta es referrer. La cabecera Referrer-Policy no contiene esta falta.

Referrer-Policy: no-referrer Referrer-Policy: no-referrer-when-downgrade Referrer-Policy: origin Referrer-Policy: origin-when-cross-origin Referrer-Policy: same-origin Referrer-Policy: strict-origin Referrer-Policy: strict-origin-when-cross-origin Referrer-Policy: unsafe-url

Directivas

- no-referrer
La cabecera Referer se omitirá en su totalidad. No se enviará ningún dato de referente junto con las solicitudes.
- no-referrer-when-downgrade (predeterminado)
Este es el comportamiento predeterminado del agente de usuario si no se especifica ninguna directiva. El origen se enviará como referente cuando el nivel de seguridad del protocolo permanece igual (HTTPS -> HTTPS), pero no se enviará a destinos menos seguros (HTTPS -> HTTP).
- origin
Se enviará únicamente el origen del documento como referente en todos los casos. El documento https://ejemplo.com/pagina.html enviará el referente https://ejemplo.com/.
- origin-when-cross-origin
Se enviará un URL completo al realizarse una solicitud de origen equivalente, pero únicamente el origen para otros casos.
- same-origin
Se enviará un referente para orígenes de sitio equivalente, pero las solicitudes de origen transversal no contendrán ningún dato de referente.
- strict-origin
Solo se enviará el origen del documento como referente a destinos que a priori son igual de seguros (HTTPS -> HTTPS), pero no lo recibirán destinos menos seguros (HTTPS -> HTTP).
- strict-origin-when-cross-origin
Se enviará un URL completo al realizarse una solicitud de origen equivalente, se enviará únicamente el origen del documento a destinos igual de seguros a priori (HTTPS -> HTTPS) y no se enviará ninguna cabecera a destinos menos seguros (HTTPS -> HTTP).
- unsafe-url
Se enviará un URL completo al realizarse una solicitud de origen equivalente o de origen transversal.

> [!NOTE]
> Esta directiva filtrará los orígenes y las rutas de acceso de recursos protegidos por TLS a orígenes inseguros. Estudie atentamente el impacto resultante de esta configuración.

Ejemplos

Directiva	Documento	Navegación a	Referente
no-referrer	<https://ejemplo.com/pagina.html>	cualquier dominio o ruta de acceso	ningún referente
no-referrer-when-downgrade	<https://ejemplo.com/pagina.html>	<https://ejemplo.com/otrapagina.html>	<https://ejemplo.com/pagina.html>
no-referrer-when-downgrade	<https://ejemplo.com/pagina.html>	<https://mozilla.org>	<https://ejemplo.com/pagina.html>
no-referrer-when-downgrade	<https://ejemplo.com/pagina.html>	<http://ejemplo.org>	ningún referente
origin	<https://ejemplo.com/pagina.html>	cualquier dominio o ruta de acceso	<https://ejemplo.com/>
origin-when-cross-origin	<https://ejemplo.com/pagina.html>	<https://ejemplo.com/otrapagina.html>	<https://ejemplo.com/pagina.html>
origin-when-cross-origin	<https://ejemplo.com/pagina.html>	<https://mozilla.org>	<https://ejemplo.com/>
origin-when-cross-origin	<https://ejemplo.com/pagina.html>	<http://ejemplo.com/pagina.html>	<https://ejemplo.com/>
same-origin	<https://ejemplo.com/pagina.html>	<https://ejemplo.com/otrapagina.html>	<https://ejemplo.com/pagina.html>
same-origin	<https://ejemplo.com/pagina.html>	<https://mozilla.org>	ningún referente
strict-origin	<https://ejemplo.com/pagina.html>	<https://mozilla.org>	<https://ejemplo.com/>
strict-origin	<https://ejemplo.com/pagina.html>	<http://ejemplo.org>	ningún referente
strict-origin	<http://ejemplo.com/pagina.html>	cualquier dominio o ruta de acceso	<http://ejemplo.com/>
strict-origin-when-cross-origin	<https://ejemplo.com/pagina.html>	<https://ejemplo.com/otrapagina.html>	<https://ejemplo.com/pagina.html>
strict-origin-when-cross-origin	<https://ejemplo.com/pagina.html>	<https://mozilla.org>	<https://ejemplo.com/>
strict-origin-when-cross-origin	<https://ejemplo.com/pagina.html>	<http://example.org>	ningún referente
unsafe-url	<https://ejemplo.com/pagina.html?q=123>	cualquier dominio o ruta de acceso	<https://ejemplo.com/pagina.html?q=123>

Especificaciones

Compatibilidad con navegadores

Véase también

- Referente HTTP en Wikipedia
- Otras maneras de definir una directiva de referentes:
- Un elemento con un nombre de referrer.
- Un atributo referrerpolicy en un elemento , , , o .
- La relación de enlace noreferrer en un elemento a, area o link (rel="noreferrer").
- Al utilizar Fetch: Request.referrerPolicy

- Directiva de origen equivalente
- «Un mayor control sobre sus referentes» en el blog de seguridad de Mozilla

Access-Control-Allow-Origin

El encabezado de respuesta Access-Control-Allow-Origin indica si los recursos de la respuesta pueden ser compartidos con el dado.

<tbody>

Tipo de encabezado |

Response header
|

Forbidden header name
|
no |

</tbody>

Sintaxis

Access-Control-Allow-Origin: * Access-Control-Allow-Origin: <origen> Access-Control-Allow-Origin: null

Directivas

- *
Para las peticiones sin credenciales, el servidor puede especificar el caracter "*" como un comodín, permitiendo a cualquier origen acceder al recurso. El acceso será permitido solamente para las peticiones hechas con el atributo crossorigin definido como "anonymous". Intentar usar el comodín con credenciales resultará en un error.
- <origen>
Especifica que origen puede acceder al recurso. Sólo se puede especificar un origen.

Ejemplos

Para permitir a cualquier origen el acceso a tus recursos, puedes especificar:

Access-Control-Allow-Origin: *

Una respuesta que le dice al navegador que permita la petición de código del origen https://developer.mozilla.org para acceder a los recursos que incluyan lo siguiente:

Access-Control-Allow-Origin: https://developer.mozilla.org

Limitando los posibles valores Access-Control-Allow-Origin de un conjunto de orígenes permitidos requiere código del lado del servidor para revisar el valor de la encabezado de petición Origin, comparan con la lista de valores permitidos, y entonces si el valor Origin se encuentra en la lista, para definir el valor de Access-Control-Allow-Origin al mismo valor que Origin.

CORS y caché

Si el servidor envía una respuesta con un valor Access-Control-Allow-Origin que es un origen explícito (en lugar del comodín "*"), entonces a respuesta debería incluir también el encabezado de respuesta Vary con el valor origin - para indicar a los navegadores que las respuestas del servidor pueden diferir basadas en el valor del encabezado de respueta Origin.

Access-Control-Allow-Origin: https://developer.mozilla.org Vary: Origin

Especificaciones

Compatibilidad con navegadores

Veáse también

- Origin
- Vary
- Cross-Origin Resource Sharing (CORS)